NLP TP REPORT

Petar Majić
January 2025

TP1 - 2
Part 1: Question answering

1. Explain the different steps used in the notebook help_Preprocessing_NLP.ipynb

- **Lowercasing**: each character is converted to lowercase to reduce vocabulary size.
- **Replace noise**: URLs are eliminated from the text using simple regular expressions (regex).
- **Removing punctuation**: punctuation is removed using regex that deletes everything that is not alphanumeric and not whitespace.
- **Tokenization**: word tokenization that divides the text into words using the NLTK library.
- **Stop-word removal**: removing words that are common terms in a language - these words are considered the least informative.
- **Stemming**: removing prefixes and suffixes of a word, the result of this process is the stem - different forms of a noun, verb or adverb share the same stem.
- **Lemmatization**: converting words into their dictionary form (lemma) using linguistic rules and vocabulary.

2. What is the main difference between stemming and lemmatization? Where are these methods applicable?

Stemming is a simple process that cuts word endings to obtain a stem. It is fast but can produce non-real words. It is mainly used for information retrieval and large text classification tasks.

Lemmatization converts words into their dictionary form (lemma) using linguistic rules. It is more accurate but slower. Iti is used in tasks that require better semantic understanding like question answering.

3. 3. What is the difference between the methods used in the notebooks help_LSAsimple_NLP.ipynb and help_Word2Vec_NLP.ipynb?

 LSA (Latent Semantic Analysis)
LSA is a **statistical method based** on a term-document matrix and Singular Value Decomposition (SVD)**.** It reduces the dimensionality of the data and captures latent semantic relationships between words and documents. It is a count-based and linear method.

 Word2Vec:
Word2Vec is a **neural network–based** method that learns word representations by predicting words from their context. It produces dense vector embeddings where semantically similar words have similar vectors. It is a predictive model trained on local word contexts.

Part 2: Medical notes classification using standard ML approaches

In this experiment I evaluated the impact of three preprocessing steps Lowercasing (LOW), Stop-word Removal (RSW), Stemming (STM) and their combinations using a TF-IDF + SVM (LinearSVC) classifier.

- **LOW (Lowercasing)**
  Accuracy: **0.73**
  Lowercasing reduces vocabulary size and gives stable baseline performance.
- **RSW (Remove Stop Words)**
  Accuracy: **0.73**
  Removing stop words does not significantly change the performance compared to LOW.
- **STM (Stemming)**
  Accuracy: **0.73**
  Stemming reduces word variation but does not improve classification accuracy.
- **LOW + RSW**
  Accuracy: **0.73**
  Combining lowercasing and stop-word removal produces similar results to using each step alone.
- **LOW + STM**
  Accuracy: **0.73**
  Adding stemming to lowercasing does not bring additional improvement.
- **RSW + STM**
  Accuracy: **0.73**
  More aggressive preprocessing still leads to the same overall performance.
- **LOW + RSW + STM**
  Accuracy: **0.73**
  Applying all preprocessing steps together does not improve results and may remove useful information for this specific task.

Best preprocessing and comparison with LSA and Word2Vec

- **Best** preprocessing approach**:**
  LOW and LOW + RSW are the best choices because they achieve the same accuracy **(**0.73**)** while keeping preprocessing simple.
- Comparison with **LSA**:
  It captures global semantic patterns but does not significantly outperform TF-IDF with simple preprocessing.
- Comparison with **Word2Vec**:
  It captures semantic similarity but averaging word vectors can lose important information.

Simple TF-IDF with minimal preprocessing performs competitively. More complex techniques such as LSA and Word2Vec do not clearly outperform the baseline in this experiment.

TP4

The goal of this practical course was to build a medical text classification pipeline. The main objective was to automatically classify medical notes into their correct medical specialties using natural language processing methods. During this project, I first implemented a classical machine learning approach using TF-IDF and SVM. Then, I also tested a Transformer-based model (BERT) in order to compare their performance, cost, and usefulness in a medical context.

The dataset used in this project is the Medical Notes 2019 dataset from Kaggle. It contains medical reports stored as individual text files with a CSV file that links each report to a medical specialty. Each note belongs to a domain Orthopedic, Radiology, Gastroenterology, Neurology, or Urology. The dataset was loaded by reading the labels file and then opening each text file using its identifier. After loading, each data sample contained an ID, a label, and the full text of the medical note.

Before training the models, the dataset was analyzed. I observed that the medical notes are long, unstructured, and written using medical terminology. Many notes include formatting elements such as HTML tags and extra spaces. I noticed that some medical specialties appear more often than others so the dataset is not perfectly balanced. This imbalance can affect the classification results and should be taken into account when analyzing the model performance.

To prepare the data for training, a simple text preprocessing step was applied. HTML tags were removed and unnecessary spaces were cleaned from the text. The goal was not to heavily modify the text, but only to remove noise while keeping important medical information. The cleaned text was stored in a new column and used as input for the models.

As a baseline solution, we used a TF-IDF vectorizer combined with a Support Vector Machine classifier. TF-IDF transforms the cleaned medical notes into numerical vectors by giving more importance to informative words and less importance to very common words. A linear SVM was then trained on these vectors to classify the notes into medical specialties. The dataset was split into training and test sets, and the model was evaluated using accuracy, precision, recall, F1-score, and a confusion matrix.

The TF-IDF + SVM model achieved an accuracy of around 71 to 72 percent. Some medical specialties, such as Orthopedic and Gastroenterology, were classified quite well. Other specialties, such as Neurology and Radiology, were more difficult to distinct. This is likely because different medical domains often use similar vocabulary and writing styles. Even though the model is simple, it provided good results and served as a strong baseline.

In the second part of the project, we fine-tuned a BERT model for medical text classification. In our experiments, the fine-tuned BERT model did **not outperform the TF-IDF + SVM baseline**. The obtained accuracy was lower than the baseline, while the training time was significantly longer. Fine-tuning BERT required much more computational resources and time, even when using only one epoch and reduced training settings. This shows that, for this dataset, the

performance gain of BERT does not justify its high computational cost. The results suggest that BERT is not always the best choice for medical text classification, especially when the dataset is limited and fast, efficient solutions are required.

In healthcare, natural language processing is useful because it helps process large amounts of medical text quickly. However, using NLP in medicine also comes with important risks. One major issue is interpretability: medical decisions must be understandable and complex models like Transformers are difficult to explain. There is also a risk of overfitting especially when datasets are small or unbalanced which can lead to unreliable predictions. Deployment cost and energy consumption are also important concerns, since large models such as BERT require powerful hardware and long training times. In medical applications, these costs are not always justified. There are also risks such as bias, data privacy issues, and the possibility of incorrect predictions affecting patient care. Because of these risks, simpler and more interpretable alternatives to BERT are often preferred. These alternatives include TF-IDF combined with classical classifiers like SVM or Logistic Regression, as well as lighter models such as FastText or smaller Transformer models. These approaches are faster, cheaper to deploy, and often sufficient for medical text classification tasks.